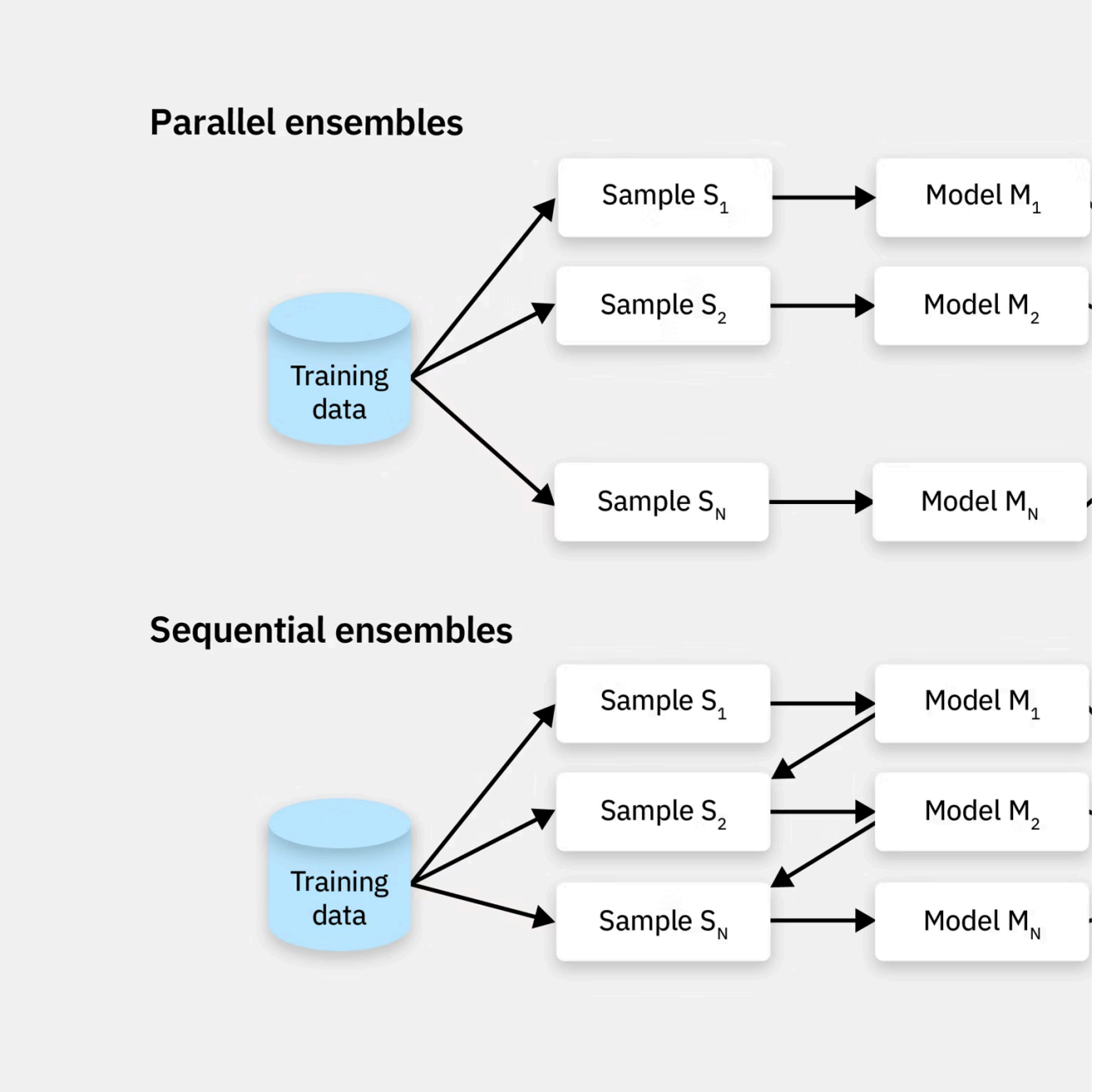
Ensemble

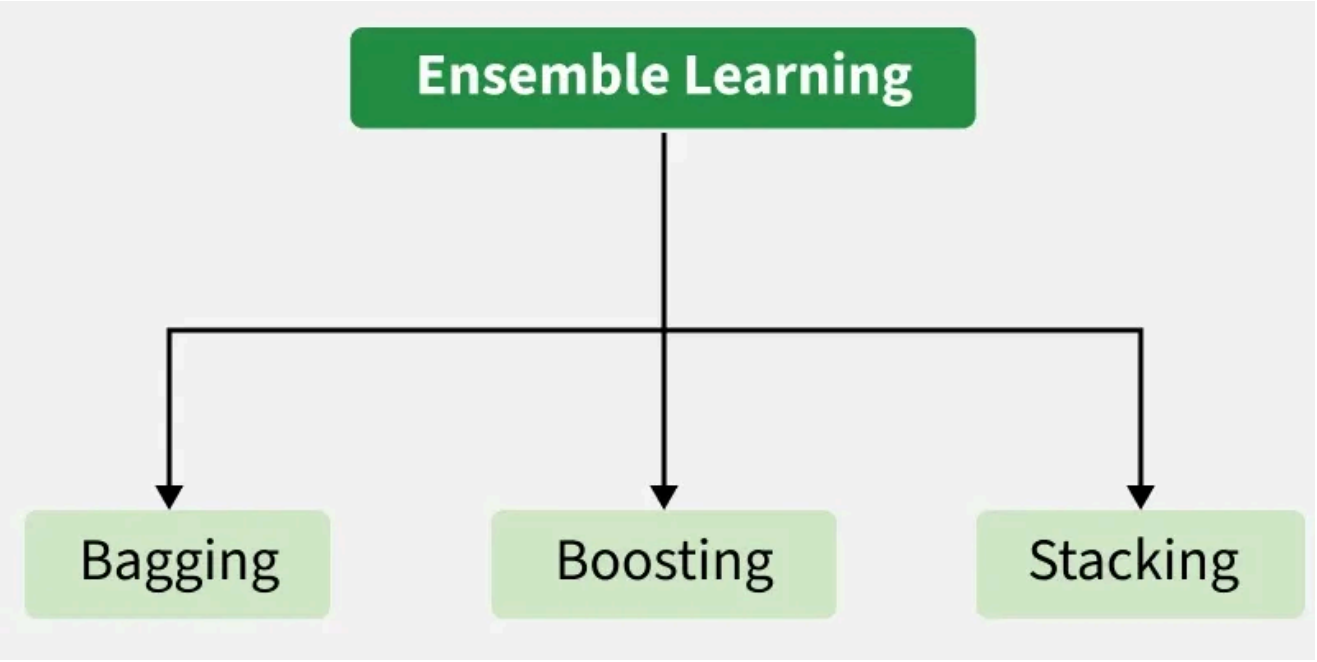
Ensemble learning is a method where we use many **small models instead of just one**.
---> **Each of these model** may not **be very strong on its own**, but when we put their results together, we get a **better** and more **accurate answer**.
Hand-on ML:
Ensemble = a group of predictors.
The ensemble learning algorithm is called an **Ensemble Method**.

The main reason why algorithms such as **random forest, extra trees, gradient boosting,** etc., are called **ensemble** is because a **final model** is generated based on many **individual models**.

From IBM
Group of Ensemble Learning
- **Parallel** methods train each base learner apart from the others of the others. Per its name, then, parallel ensembles train base learners in parallel and independent of one another.
- **Sequential** methods train a new base learner so that it minimizes errors made by the previous model trained in the preceding step. In other words, sequential methods construct base models sequentially in stages.


Parallel ensembles
Training data
Sample S_1
Sample S_2
Sample S_N
Model M_1
Model M_2
Model M_N
Sequential ensembles
Training data
Sample S_1
Sample S_2
Sample S_N
Model M_1
Model M_2
Model M_N

From G4G


Ensemble Learning
Bagging
Boosting
Stacking

Type of Ensemble learning:

1. **Bagging (Bootstrap Aggregating)**: **Models** are **trained independently** on **different random subsets of the training data**. Their results are then combined—usually by **averaging** (for regression) or **voting** (for classification).

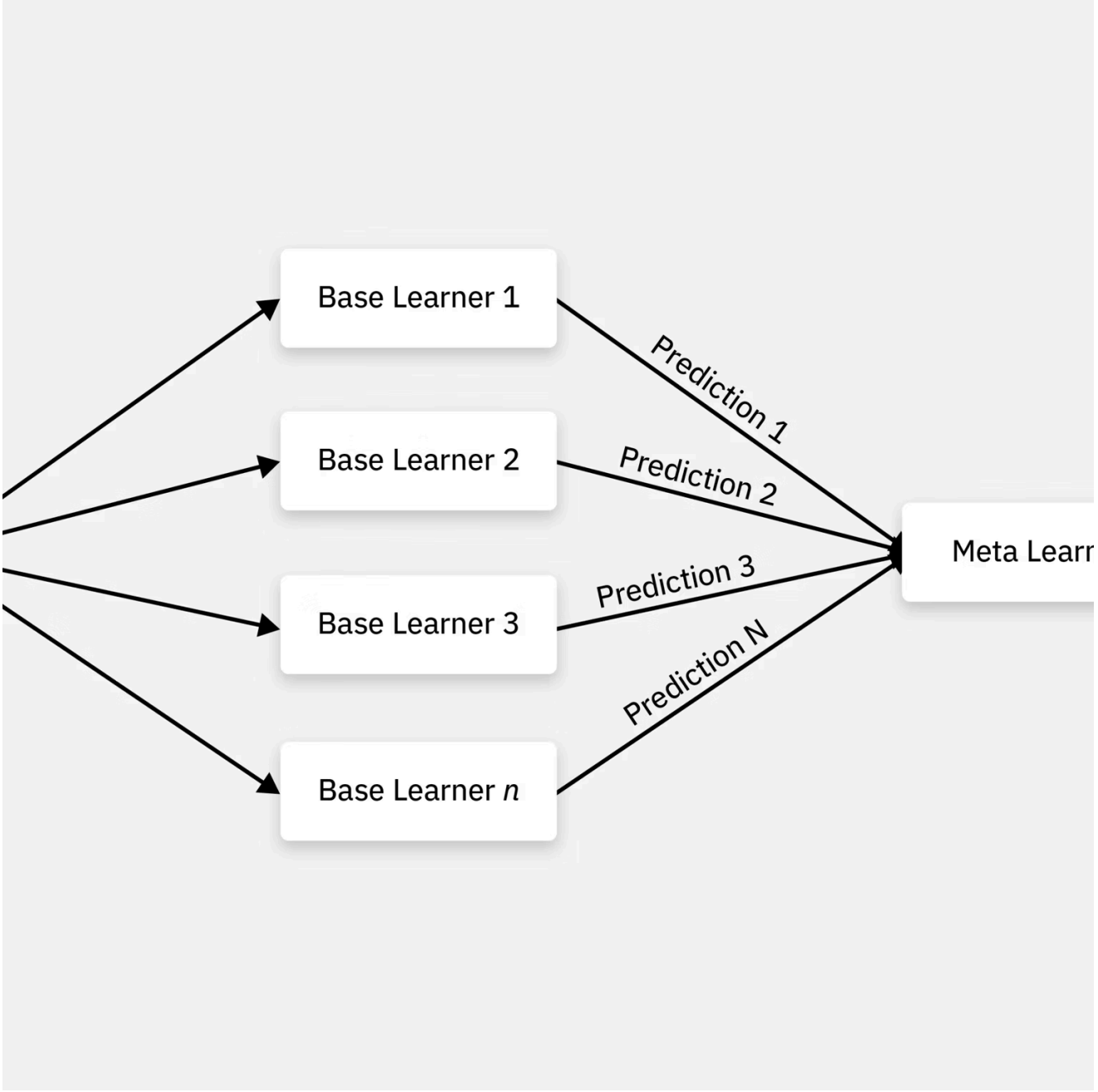
--> This helps reduce **variance** and prevents **overfitting**.

2. Boosting**: Models** are **trained one after another**. Each new model focuses on fixing the errors made by the previous ones. The **final prediction** is a **weighted combination** of **all models**.

--> This helps reduce **bias** and improve **accuracy**.

3. **Stacking (Stacked Generalization)**: Multiple different models (often of different types) are trained and their predictions are used as inputs to a final model, called a **meta-model**. The meta-model learns how to best combine the predictions of the base models, aiming for better performance than any individual model.

Stacking



Bagging and Boosting are widely use

Base Models: Individual models that form the ensemble. For example, Support Vector Machines, Logistic Regression, Decision Trees.

Voting = it is an ensemble technique that combines **predictions** from multiple models to improve accuracy, robustness, and generalization.
it combines multiple individual classifiers or regressor to make predictions. it support both hard and soft voting strategies.

- **Hard Voting**: The class that receives the majority of votes is selected as the final prediction. It is commonly used in **classification** problems. In regression, it predicts the **average** of the individual predictions.
- **Soft Voting**: Weighted average of predicted probabilities is used to make the final prediction. It is suitable when classifiers provide probability estimates. In other words, for each class, it **sums the predicted probabilities** and **predicts the class with the highest sum.** the class with the highest average probability is selected. this performs better than the hard voting as it considers confidence levels.
- **Weighted Voting:** instead of equal votes, weights are assigned to each model's prediction based on their performance or reliability.

Voting Classifier

```
# Step 1 - Importing libraries
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Step 2 - Loading Iris dataset
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)

# Step 3 - Define base models
model1 = LogisticRegression()
model2 = DecisionTreeClassifier()
model3 = SVC(probability=True)

# Step 4 - Creating a VotingClassifier with soft voting
voting_classifier = VotingClassifier(
    estimators=[('lr', model1), ('dt', model2), ('svc', model3)], voting='soft')
```

```
# Step 5 - Fit the model
voting_classifier.fit(X_train, y_train)

# Step 6 - Make predictions
y_prediction = voting_classifier.predict(X_test)

# Step 7 - Evaluating accuracy
accuracy = accuracy_score(y_test, y_prediction)
print('Accuracy:', accuracy) 95%++ performance accuracy.
```

Voting Regressor

```
# Import libraries
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.ensemble import VotingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

# Load Boston Housing dataset
boston = load_boston()
X_train, X_test, y_train, y_test = train_test_split(
    boston.data, boston.target, test_size=0.2, random_state=42)

# Choose and define base regressor models
model1 = LinearRegression()
model2 = DecisionTreeRegressor()
model3 = RandomForestRegressor()

# Create a VotingRegressor
voting_regressor = VotingRegressor(
    estimators=[('lr', model1), ('dt', model2), ('rf', model3)])

# Fit the model
voting_regressor.fit(X_train, y_train)

# Make predictions on the test set
y_prediction = voting_regressor.predict(X_test)

#Evaluate performance using Mean Squared Error (mse)
mse = mean_squarred_error(y_test, y_prediction)
print(f'Mean Squared Error: {mse: .2f}')
```